

DOCUMENTO DE APOIO

Introdução à Programação com Portugol Studio

Conceitos, exemplos de uso, códigos e saídas esperadas

Objetivo do material

Servir como referência durante as aulas e atividades, mostrando o que cada recurso faz, quando pode ser usado e como aparece em um programa.

Material para estudantes iniciantes

Sumário

1. Como utilizar este material
2. Preparação e abertura do Portugol Studio
3. Estrutura básica de um programa
4. Entrada, processamento e saída
5. Variáveis e tipos de dados
6. Entrada e saída de dados
7. Operações matemáticas
8. Operadores relacionais e lógicos
9. Estruturas de controle
10. Estruturas de repetição
11. Funções e procedimentos
12. Testes, depuração e erros comuns
13. Exemplo integrador: notas e frequência
14. Guia rápido de consulta

1. Como utilizar este material

Este documento foi preparado para acompanhar as aulas de lógica de programação. Os exemplos estão organizados do mais simples ao mais completo. Em cada tópico, você encontrará:

- o conceito principal;
- quando o recurso pode ser utilizado;
- um exemplo em Portugol Studio;
- a saída esperada;
- observações e erros comuns.

Sugestão de estudo

Digite cada exemplo no Portugol Studio, execute, observe a saída e faça pequenas alterações. A prática é essencial para compreender o comportamento do código.

2. Preparação e abertura do Portugol Studio

1. Baixe o arquivo disponibilizado pelo professor ou pela plataforma da disciplina.
2. Abra a pasta Downloads e localize o arquivo recebido.
3. Caso o arquivo esteja no formato ZIP, clique com o botão direito e escolha Extrair tudo.
4. Abra a pasta criada após a extração.
5. Localize o arquivo identificado pelo sistema como aplicativo ou executável.
6. Dê dois cliques para abrir o Portugol Studio.
7. Crie um novo programa, execute um exemplo e salve o arquivo.

Atenção

Não mova apenas o arquivo executável para fora da pasta. Alguns programas precisam dos demais arquivos que acompanham o pacote.

3. Estrutura básica de um programa

Todo programa precisa de uma estrutura inicial. No Portugol Studio, a execução começa na função início.

Exemplo 1 - Primeiro programa

```
programa
{
    funcao inicio()
    {
        escreva("Olá, mundo!")
    }
}
```

Saída esperada

Olá, mundo!

Elemento	Função
programa	Delimita o programa principal.
{ }	As chaves delimitam blocos de instruções.
funcao inicio()	Indica o ponto em que a execução começa.
escreva()	Exibe mensagens e valores na tela.
"texto"	As aspas delimitam uma cadeia de caracteres.

Quando usar

A estrutura básica aparece em todos os programas. As instruções iniciais ficam dentro da função início.

4. Entrada, processamento e saída

Um programa geralmente recebe dados, realiza alguma operação e apresenta um resultado.

ENTRADA	PROCESSAMENTO	SAÍDA
---------	---------------	-------

Exemplo: receber duas notas, calcular a média e apresentar o resultado.

Exemplo 2 - Entrada, processamento e saída

```
programa
{
    funcao inicio()
    {
        real nota1, nota2, media

        escreva("Digite a primeira nota: ")
        leia(nota1)

        escreva("Digite a segunda nota: ")
        leia(nota2)
```

```
        media = (nota1 + nota2) / 2.0  
  
        escreva("Média: ", media)  
    }  
}
```

Saída esperada

Digite a primeira nota: 8
Digite a segunda nota: 6
Média: 7.0

5. Variáveis e tipos de dados

Uma variável é um espaço identificado por um nome e utilizado para armazenar um valor. O tipo de dado determina que espécie de informação poderá ser guardada.

Tipo	O que armazena
inteiro	Números sem casas decimais, como 10, -3 e 2026.
real	Números com casas decimais, como 7.5 e 1500.90.
cadeia	Textos com vários caracteres, como nomes e frases.
caracter	Um único símbolo, letra ou dígito.
logico	Os valores verdadeiro ou falso.

Exemplo 3 - Declaração e uso de variáveis

```

programa
{
    funcao inicio()
    {
        cadeia nome = "Ana"
        inteiro idade = 19
        real media = 8.5
        caracter turma = 'A'
        logico aprovado = verdadeiro

        escreva("Nome: ", nome, "\n")
        escreva("Idade: ", idade, "\n")
        escreva("Média: ", media, "\n")
        escreva("Turma: ", turma, "\n")
        escreva("Aprovado: ", aprovado)
    }
}

```

Saída esperada

```

Nome: Ana
Idade: 19
Média: 8.5
Turma: A
Aprovado: verdadeiro

```

5.1 Regras práticas para nomes de variáveis

- utilize nomes que indiquem o conteúdo, como idade, media e valor_total;
- não use espaços no nome;
- evite iniciar o nome com número;
- prefira letras minúsculas e o caractere sublinhado para separar palavras;
- não use palavras reservadas da linguagem, como programa, se ou funcao.

Exemplos de bons nomes

nome_aluno, quantidade_produtos, media_final, possui_desconto e valor_parcela.

6. Entrada e saída de dados

6.1 Comando escreva

O comando escreva mostra textos, variáveis e resultados.

```
escreva("Total: R$ ", total)
```

Saída esperada

Total: R\$ 150.0

6.2 Comando leia

O comando leia recebe uma informação digitada pelo usuário e armazena em uma variável.

Exemplo 4 - Leitura de texto

```
programa
{
    funcao inicio()
    {
        cadeia nome

        escreva("Digite seu nome: ")
        leia(nome)

        escreva("Olá, ", nome, "!")
    }
}
```

Saída esperada

Digite seu nome: Carlos
Olá, Carlos!

7. Operações matemáticas

Os operadores aritméticos permitem realizar cálculos com números armazenados em variáveis.

Operador	Uso
+	Adição
-	Subtração
*	Multiplicação
/	Divisão
%	Resto da divisão

7.1 Soma e média

```
real n1 = 7.0
real n2 = 9.0
real media = (n1 + n2) / 2.0

escreva("Média: ", media)
```

Saída esperada

Média: 8.0

7.2 Conversão de Celsius para Fahrenheit

```
programa
{
    funcao inicio()
    {
        real celsius, fahrenheit

        escreva("Temperatura em Celsius: ")
        leia(celsius)

        fahrenheit = (celsius * 9.0 / 5.0) + 32.0

        escreva("Temperatura em Fahrenheit: ", fahrenheit)
    }
}
```

Saída esperada

Temperatura em Celsius: 25
Temperatura em Fahrenheit: 77.0

7.3 Cálculo de desconto

```
programa
{
    funcao inicio()
```

```

{
    real preco, percentual, desconto, total

    escreva("Preço do produto: ")
    leia(preco)

    escreva("Percentual de desconto: ")
    leia(percentual)

    desconto = preco * percentual / 100.0
    total = preco - desconto

    escreva("Desconto: R$ ", desconto, "\n")
    escreva("Total: R$ ", total)
}

```

Saída esperada

Preço do produto: 200
 Percentual de desconto: 10
 Desconto: R\$ 20.0
 Total: R\$ 180.0

7.4 Juros simples

```

programa
{
    funcao inicio()
    {
        real capital, taxa, juros, montante
        inteiro meses

        escreva("Capital: ")
        leia(capital)

        escreva("Taxa mensal em porcentagem: ")
        leia(taxa)

        escreva("Quantidade de meses: ")
        leia(meses)

        juros = capital * (taxa / 100.0) * meses
        montante = capital + juros

        escreva("Juros: R$ ", juros, "\n")
        escreva("Montante: R$ ", montante)
    }
}

```

Saída esperada

Capital: 1000
 Taxa mensal em porcentagem: 2
 Quantidade de meses: 3
 Juros: R\$ 60.0
 Montante: R\$ 1060.0

8. Operadores relacionais e lógicos

8.1 Operadores relacionais

Operador	Significado
>	Maior que
<	Menor que
>=	Maior ou igual
<=	Menor ou igual
==	Igual
!=	Diferente

8.2 Operadores lógicos

Operador	Quando o resultado é verdadeiro
e	Quando todas as condições são verdadeiras.
ou	Quando pelo menos uma condição é verdadeira.
nao	Inverte o resultado lógico.

```
real media = 7.5
real frequencia = 80.0

logico aprovado = media >= 7.0 e frequencia >= 75.0

escreva("Aprovado: ", aprovado)
```

Saída esperada

Aprovado: verdadeiro

Dica

Use parênteses para deixar expressões longas mais claras. Exemplo: (media >= 7.0) e (frequencia >= 75.0).

9. Estruturas de controle

As estruturas de controle permitem alterar o fluxo do programa. Elas são usadas para tomar decisões.

9.1 Condicional simples - se

Use quando um bloco deve executar apenas se a condição for verdadeira.

```
programa
{
    funcao inicio()
    {
        inteiro idade

        escreva("Digite a idade: ")
        leia(idade)

        se (idade >= 18)
        {
            escreva("Maior de idade")
        }
    }
}
```

Saída esperada

Digite a idade: 20
Maior de idade

9.2 Condicional composta - se e senao

Use quando existem dois caminhos possíveis.

```
se (media >= 7.0)
{
    escreva("Aprovado")
}
senao
{
    escreva("Não aprovado")
}
```

Saída para media = 8.0

Aprovado

9.3 Condicionais encadeadas

Use quando existem três ou mais classificações.

```
se (media >= 7.0)
{
    escreva("Aprovado")
}
senao se (media >= 5.0)
{
    escreva("Recuperação")
}
senao
{
    escreva("Reprovação")
}
```

```
    escreva("Reprovado")
}
```

Saída para media = 6.0

Recuperação

9.4 Escolha e caso

Use quando uma opção inteira ou caractere deve selecionar uma entre várias alternativas.

```
programa
{
    funcao inicio()
    {
        inteiro opcao

        escreva("1 - Cadastrar\n")
        escreva("2 - Consultar\n")
        escreva("3 - Sair\n")
        escreva("Escolha: ")
        leia(opcao)

        escolha (opcao)
        {
            caso 1:
                escreva("Cadastro selecionado")
                pare

            caso 2:
                escreva("Consulta selecionada")
                pare

            caso 3:
                escreva("Programa encerrado")
                pare

            caso contrario:
                escreva("Opção inválida")
        }
    }
}
```

Saída esperada

```
1 - Cadastrar
2 - Consultar
3 - Sair
Escolha: 2
Consulta selecionada
```

10. Estruturas de repetição

As repetições executam um bloco várias vezes. A escolha depende de como a quantidade de repetições será controlada.

10.1 enquanto

Use quando a repetição depende de uma condição verificada antes da execução.

```
programa
{
    funcao inicio()
    {
        inteiro contador = 1

        enquanto (contador <= 5)
        {
            escreva(contador, "\n")
            contador = contador + 1
        }
    }
}
```

Saída esperada

```
1
2
3
4
5
```

10.2 faca-enquanto

Use quando o bloco precisa executar pelo menos uma vez, como em validações de entrada.

```
programa
{
    funcao inicio()
    {
        real nota

        faca
        {
            escreva("Digite uma nota entre 0 e 10: ")
            leia(nota)
        }
        enquanto (nota < 0.0 ou nota > 10.0)

            escreva("Nota válida: ", nota)
    }
}
```

Saída esperada

```
Digite uma nota entre 0 e 10: 15
Digite uma nota entre 0 e 10: 8
Nota válida: 8.0
```

10.3 para

Use quando a quantidade de repetições é conhecida.

```

programa
{
    funcao inicio()
    {
        para (inteiro i = 1; i <= 5; i++)
        {
            escreva("Repetição ", i, "\n")
        }
    }
}

```

Saída esperada

```

Repetição 1
Repetição 2
Repetição 3
Repetição 4
Repetição 5

```

10.4 Contador e acumulador

O contador registra quantas vezes algo ocorreu. O acumulador soma valores ao longo das repetições.

```

programa
{
    funcao inicio()
    {
        inteiro contador
        real nota, soma = 0.0

        para (contador = 1; contador <= 3; contador++)
        {
            escreva("Nota ", contador, ": ")
            leia(nota)

            soma = soma + nota
        }

        escreva("Média: ", soma / 3.0)
    }
}

```

Saída esperada

```

Nota 1: 7
Nota 2: 8
Nota 3: 9
Média: 8.0

```

11. Funções e procedimentos

Funções e procedimentos organizam o programa em blocos menores. Isso facilita a reutilização, a leitura e os testes.

11.1 Função com retorno

Use quando o bloco precisa calcular e devolver um valor.

```
programa
{
    funcao real calcular_media(real n1, real n2)
    {
        retorne (n1 + n2) / 2.0
    }

    funcao inicio()
    {
        real resultado = calcular_media(8.0, 6.0)

        escreva("Média: ", resultado)
    }
}
```

Saída esperada

Média: 7.0

11.2 Procedimento sem retorno

Use quando o bloco apenas executa uma tarefa, sem devolver um valor.

```
programa
{
    funcao mostrar_linha()
    {
        escreva("-----\n")
    }

    funcao inicio()
    {
        mostrar_linha()
        escreva("MENU PRINCIPAL\n")
        mostrar_linha()
    }
}
```

Saída esperada

```
-----
MENU PRINCIPAL
-----
```

11.3 Função para classificar resultado

```
programa
{
    funcao cadeia classificar(real media)
    {
        se (media >= 7.0)
```

```
{
    retorne "Aprovado"
}
senao se (media >= 5.0)
{
    retorne "Recuperação"
}
senao
{
    retorne "Reprovado"
}
}

funcao inicio()
{
    escreva(classificar(6.5))
}
}
```

Saída esperada

Recuperação

12. Testes, depuração e erros comuns

Depurar significa localizar, compreender e corrigir problemas. Um código que executa sem apresentar mensagem de erro ainda pode produzir um resultado incorreto.

Problema	Como verificar ou corrigir
Variável não inicializada	Atribua um valor antes de utilizar a variável.
Divisão inteira inesperada	Use valores reais, como 2.0, quando precisar de resultado decimal.
Laço infinito	Confirme se a variável de controle é atualizada.
Condição invertida	Teste valores abaixo, iguais e acima do limite.
Entrada inválida	Valide o dado com faça-enquanto.
Chaves incompletas	Confira a abertura e o fechamento de cada bloco.
Texto sem aspas	Coloque mensagens entre aspas duplas.
Comparação escrita com =	Use == para comparar e = para atribuir.

12.1 Modelo de teste de mesa

Passo	Valor lido	Variáveis após o passo	Saída
1	8	nota1 = 8	
2	6	nota2 = 6	
3	-	media = 7	
4	-	media = 7	Média: 7

Teste mínimo recomendado

Teste um valor normal, um valor exatamente no limite e um valor inválido. Em decisões, teste todos os caminhos possíveis.

13. Exemplo integrador: notas e frequência

O programa abaixo combina variáveis, entrada, cálculos, validação, condicionais, repetições, contadores e funções.

Projeto de referência

```

programa
{
    funcao real ler_nota(cadeia mensagem)
    {
        real nota

        faca
        {
            escreva(mensagem)
            leia(nota)
        }
        enquanto (nota < 0.0 ou nota > 10.0)

        retorne nota
    }

    funcao real ler_frequencia()
    {
        real frequencia

        faca
        {
            escreva("Frequência de 0 a 100: ")
            leia(frequencia)
        }
        enquanto (frequencia < 0.0 ou frequencia > 100.0)

        retorne frequencia
    }

    funcao cadeia classificar(real media, real frequencia)
    {
        se (frequencia < 75.0)
        {
            retorne "Reprovado por frequência"
        }
        senao se (media >= 7.0)
        {
            retorne "Aprovado"
        }
        senao se (media >= 5.0)
        {
            retorne "Recuperação"
        }
        senao
        {
            retorne "Reprovado"
        }
    }

    funcao inicio()
    {
        inteiro quantidade, aprovados = 0, recuperacao = 0
        inteiro reprovados = 0, reprovados_frequencia = 0
        real soma_medias = 0.0

        escreva("Quantidade de estudantes: ")
        leia(quantidade)
    }
}

```

```

para (inteiro i = 1; i <= quantidade; i++)
{
    cadeia nome, situacao
    real n1, n2, media, frequencia

    escreva("\nNome do estudante: ")
    leia(nome)

    n1 = ler_nota("Nota 1: ")
    n2 = ler_nota("Nota 2: ")
    frequencia = ler_frequencia()

    media = (n1 + n2) / 2.0
    situacao = classificar(media, frequencia)

    soma_medias = soma_medias + media

    se (situacao == "Aprovado")
    {
        aprovados++
    }
    senao se (situacao == "Recuperação")
    {
        recuperacao++
    }
    senao se (situacao == "Reprovado por frequência")
    {
        reprovados_frequencia++
    }
    senao
    {
        reprovados++
    }

    escreva("Média: ", media, "\n")
    escreva("Situação: ", situacao, "\n")
}

escreva("\nRESUMO DA TURMA\n")
escreva("Aprovados: ", aprovados, "\n")
escreva("Recuperação: ", recuperacao, "\n")
escreva("Reprovados: ", reprovados, "\n")
escreva("Reprovados por frequência: ", reprovados_frequencia, "\n")

se (quantidade > 0)
{
    escreva("Média geral: ", soma_medias / quantidade)
}
}

```

Saída esperada

Quantidade de estudantes: 2

Nome do estudante: Ana
 Nota 1: 8
 Nota 2: 7
 Frequência de 0 a 100: 90
 Média: 7.5
 Situação: Aprovado

Nome do estudante: Bruno

```
Nota 1: 6
Nota 2: 5
Frequência de 0 a 100: 80
Média: 5.5
Situação: Recuperação
```

```
RESUMO DA TURMA
Aprovados: 1
Recuperação: 1
Reprovados: 0
Reprovados por frequência: 0
Média geral: 6.5
```

13.1 O que foi utilizado

- variáveis dos tipos inteiro, real e cadeia;
- entrada e saída com leia e escreva;
- operações matemáticas para calcular médias;
- condicionais para classificar estudantes;
- faça-enquanto para validar notas e frequência;
- para para repetir o cadastro;
- contadores para o resumo da turma;
- funções para organizar e reutilizar o código.

14. Guia rápido de consulta

Necessidade	Recurso sugerido
Guardar um número inteiro	inteiro
Guardar um valor decimal	real
Guardar um nome ou frase	cadeia
Guardar verdadeiro ou falso	logico
Mostrar uma mensagem	escreva
Receber um valor	leia
Executar algo quando uma condição for verdadeira	se
Escolher entre dois caminhos	se e senao
Escolher entre várias opções	escolha e caso
Repetir enquanto uma condição for verdadeira	enquanto
Executar ao menos uma vez e validar depois	faca-enquanto
Repetir uma quantidade conhecida	para
Reutilizar um cálculo	funcao com retorne
Reutilizar uma sequência de comandos	funcao sem retorno

14.1 Sequência sugerida de prática

1. Execute o programa Olá, mundo.
2. Leia e mostre nome, idade e cidade.
3. Calcule média, desconto e conversão de temperatura.
4. Crie decisões com se e senao.
5. Faça classificações com condicionais encadeadas.
6. Crie um menu com escolha e caso.
7. Use enquanto, faca-enquanto e para.
8. Transforme cálculos repetidos em funções.
9. Teste casos normais, limites e entradas inválidas.
10. Desenvolva o sistema de notas e frequência.

Referências

UNIVALI-LITE. Portugol Studio. Ambiente didático para aprendizagem de programação em português.
 WING, Jeannette M. Computational Thinking. Communications of the ACM, v. 49, n. 3, p. 33-35, 2006.
 Materiais produzidos para o curso Introdução à Lógica de Programação com Portugol Studio.